

Previsão de Atrasos em Voos Comerciais: Relatório Global do Projeto — Fases 1 a 6

Francisco Palmeira 2109923

Universidade da Madeira

Engenharia Informática

Funchal, Portugal

2109923@student.uma.pt

Omar Jesus 2099223

Universidade da Madeira

Engenharia Informática

Funchal, Portugal

2099223@student.uma.pt

Resumo—Este relatório documenta o ciclo de vida completo de análise de dados — Fases 1 a 6 — aplicado ao *Flight Delay and Cancellation Dataset (2019–2023)* do Departamento de Transportes dos EUA. O problema é reformulado como classificação triclasse — *On-time* (<15 min), *Short delay* (15–30 min) e *Long delay* (>30 min) — e como regressão sobre os minutos de atraso à chegada. As Fases 1–3 estabelecem uma *pipeline* de preparação que elimina rigorosamente o *data leakage* pós-partida, cria doze novas *features* (temporais, geográficas e *target encoding*), e valida o sinal preditivo por análise exploratória (PCA, t-SNE, UMAP, importância por permutação) e testes de hipóteses (Welch e Kruskal-Wallis, $p < 0,0001$). As Fases 4–6 constroem seis famílias de modelos: KNN implementado de raiz em NumPy, modelos supervisionados clássicos (Árvore de Decisão e Regressão Logística), dois *ensembles* (Random Forest e Hist-GradientBoosting), quatro variantes MLP em TensorFlow/Keras, e agrupamento de aeroportos com K-Means, DBSCAN e Agglomerative Ward. Diagnostica-se um *bug* estrutural na interação `class_weight × EarlyStopping(val_loss)` do Keras, corrigido com um *callback* personalizado (MacroF1Monitor) que eleva o Macro F1 de 0,28 para 0,42. Um modelo hierárquico de três estágios reduz o MAE de ponta a ponta para 26,09 min e o RMSE do subconjunto *Short delay* para apenas 6,03 min. Demonstra-se que a previsão ao minuto com *features* pré-partida é fundamentalmente limitada, com $R^2 < 0$ em todos os cenários, o que motiva a formulação como classificação e, em particular, a questão binária «*vai atrasar?*» como produto operacional.

Index Terms—Ciência de Dados, Feature Engineering, Testes de Hipóteses, Classificação Multiclasse, KNN, Árvore de Decisão, Regressão Logística, Random Forest, Gradient Boosting, MLP, Deep Learning, Clustering, K-Means, DBSCAN, Agglomerative Ward, Modelo Hierárquico.

I. INTRODUÇÃO

A. Contextualização do Problema

A pontualidade é um dos indicadores de desempenho mais críticos da aviação comercial. Os atrasos geram custos operacionais elevados — combustível desperdiçado, tripulações imobilizadas, portões bloqueados e indemnizações — e degradam a satisfação dos passageiros. Este projeto utiliza dados reais do Departamento de Transportes dos EUA [1] para abordar o problema através da previsão proativa de atrasos a partir de fatores operacionais e temporais conhecidos *antes* da partida.

B. Objetivos da Análise

O objetivo central é classificar o estado de chegada de um voo em três categorias: *On-time* (atraso <15 min), *Short delay*

(15–30 min) e *Long delay* (>30 min). Em paralelo, formula-se a tarefa de regressão sobre os minutos exatos de atraso à chegada (ARR_DELAY), permitindo avaliar até que ponto o problema é resolúvel ao minuto. Esta dupla formulação alinha-se diretamente com as decisões operacionais das companhias aéreas: a decisão binária/triclasse orienta a alocação de recursos, e a estimativa em minutos quantifica a severidade esperada.

C. Conjunto de Dados

O conjunto original contém mais de **3 milhões** de registos de voos domésticos comerciais dos EUA entre 2019 e 2023. Após a preparação descrita na Secção II, o conjunto de treino final contém **200.000 voos** e o de teste **50.000 voos**. As *features* utilizadas nos modelos são exclusivamente pré-partida: CRS_DEP_TIME, CRS_ARR_TIME, CRS_ELAPSED_TIME, DISTANCE, AIRLINE_CODE, ORIGIN, DEST e as **doze features** derivadas detalhadas na Secção II (Tabela II).

II. FASE 1: PREPARAÇÃO DOS DADOS

A preparação dos dados foi implementada como uma *pipeline* modular, com uma classe Python dedicada a cada etapa (DataLoader, DataCleaning, FeatureEngineering, DataPreprocessing), orquestradas por um *script* principal com *checkpoints* em disco (pickle) entre fases para permitir reprodutibilidade e evitar recomputações dispendiosas. Esta secção documenta exaustivamente cada decisão tomada e o seu fundamento.

A. Carregamento, Balanceamento e Divisão (DataLoader)

O ponto de partida é o ficheiro `flights_sample_3m.csv`, uma amostra de ≈ 3 milhões de registos de voos domésticos comerciais dos EUA (2019–2023). Cada registo descreve um voo agendado e o seu desfecho, incluindo identificadores (companhia, aeroportos de origem e destino), horários planeados, distância, tempos reais e a decomposição do atraso por causa. Logo no carregamento são tomadas quatro decisões estruturais:

- 1) **Exclusão de voos cancelados e divergidos** (`CANCELLED == 0` e `DIVERTED == 0`), por não possuírem atraso à chegada quantificável.

- 2) **Truncamento de atrasos negativos a zero** (`ARR_DELAY.clip(lower=0)`): prever chegadas antecipadas não acrescenta valor operacional, pelo que voos adiantados são tratados como pontuais.
- 3) **Balanceamento por *undersampling***: a classe maioritária de voos *pontuais* (`ARR_DELAY == 0`) é subamostrada aleatoriamente (`random_state=42`) até igualar o número de voos *atrasados* (`ARR_DELAY > 0`), produzindo um conjunto equilibrado a **50%/50%** entre pontuais e atrasados, depois baralhado.
- 4) **Divisão treino/teste** com `train_test_split` (`test_size=0.2, random_state=42`), realizada *antes* das fases seguintes para garantir que nenhuma estatística do teste contamina o treino.

É importante notar que este balanceamento 50/50 opera sobre a partição *binária* (pontual vs. atrasado), *não* sobre as três classes finais. Como a classe *On-time* (<15 min) absorve ainda os voos com atraso entre 1 e 14 minutos, a distribuição *triclasse* resultante permanece desequilibrada em **≈73% On-time / 10% Short / 17% Long** — facto determinante para toda a fase de modelação. Para a modelação, usam-se subconjuntos de **200.000 voos de treino** e **50.000 de teste**.

B. Prevenção de Data Leakage

Assegurar a ausência de *data leakage* foi a prioridade crítica do projeto: qualquer variável conhecida apenas *após* a descolagem, ou derivada da variável alvo, inflaciona artificialmente o desempenho e invalida o modelo em produção. Removeram-se, por isso, **17 colunas**, organizadas em quatro grupos de risco distintos (Tabela I). A regra de ouro adotada foi inequívoca: o modelo só pode aceder a informação disponível no *momento do planeamento* do voo.

Tabela I: Colunas removidas por risco de *data leakage*

Grupo		Colunas	Razão
Eventos partida	pós-	DEP_TIME, DEP_DELAY, TAXI_OUT, WHEELS_OFF, WHEELS_ON, TAXI_IN, ARR_TIME	Só conhecidos depois de o voo partir
Resultados do voo	reais	ACTUAL_ELAPSED_TIME, AIR_TIME	Medidos durante/após o voo
Estado terminal		CANCELLED, CANCELLATION_CODE, DIVERTED	Desfecho operacional
Decomposição do alvo		DELAY_DUE_CARRIER, _WEATHER, _SECURITY, _LATE_AIRCRAFT, _NAS	Derivadas diretamente de ARR_DELAY

C. Limpeza de Dados (*DataCleaning*)

Já após a divisão treino/teste, a classe `DataCleaning` aplica uma sequência determinística de quatro operações. Crucialmente, as transformações que poderiam introduzir *leakage* são restritas ao conjunto de **treino**:

- 1) **Remoção das colunas de *leakage*** (Tabela I), aplicada a treino e teste.

- 2) **Remoção de registos duplicados**, apenas no **treino**, eliminando voos repetidos que enviesariam as frequências e o ajuste do modelo (o teste é preservado intacto para refletir a realidade).
- 3) **Tratamento de valores ausentes por eliminação** das linhas com qualquer valor em falta (`dropna`), em treino e teste, com realinhamento dos rótulos. Optou-se pela remoção, em vez da imputação, por o volume de dados ser amplo o suficiente para a tolerar sem perda de representatividade, evitando introduzir valores sintéticos.
- 4) **Remoção de outliers pelo método do Z-score**, apenas no **treino**: descartam-se as linhas em que *qualquer feature* numérica excede um desvio de $|z| > 2$ face à média da respetiva coluna. Note-se que este filtro incide sobre as *features* (horários, distância, tempos planeados) e **não** sobre a variável alvo `ARR_DELAY` — a cauda direita de atrasos extremos, sinal operacional genuíno, é assim preservada. O único tratamento do alvo (truncamento de negativos a zero) já ocorrera no carregamento.

A conversão de `FL_DATE` para `datetime`, necessária à extração das *features* temporais, é assegurada na fase seguinte. Em síntese, o desequilíbrio *triclasse* (73/10/17) que sobrevive ao balanceamento binário não é eliminado por nova reamostragem global nesta fase; é endereçado, de forma controlada, ao nível de cada modelo — via `class_weight` e, em experiências dedicadas, *undersampling* adicional da classe maioritária (cf. Fases 4–5).

D. Engenharia de Features

Para enriquecer a capacidade preditiva e capturar os padrões contextuais que a análise viria a confirmar como dominantes, construíram-se **doze novas features** a partir dos atributos brutos. A Tabela II documenta cada uma com a sua definição exata (tal como implementada no código) e a respetiva codificação, agrupadas em três famílias: temporais, geográficas/operacionais e *target encoding*.

A justificação destas escolhas é central para o projeto. As *features* temporais codificam a hipótese de que *quando* um voo ocorre (estação, hora, fim de semana, época de férias) condiciona fortemente o atraso esperado. A `ROUTE` e o `FLIGHT_TYPE` capturam a dimensão geográfico-operacional. Já a `AVG_DELAY_PER_HOUR`, obtida por *target encoding*, é a tentativa mais direta de injetar sinal histórico: codifica, para cada hora de partida, o atraso médio observado. **A sua construção exige cuidado anti-leakage**: o mapeamento hora → atraso médio é calculado *exclusivamente* no conjunto de treino (`groupby('CRS_DEP_HOUR')` sobre `ARR_DELAY` do treino) e só depois aplicado ao teste; horas eventualmente ausentes no teste são preenchidas com a média global do treino. Por esta dependência do alvo, esta variável é criada já dentro da fase de pré-processamento, *após* a divisão treino/teste — e nunca antes.

E. Definição da Variável Alvo

O atraso contínuo `ARR_DELAY` (em minutos) é discretizado em três classes operacionalmente significativas, mantendo-se

Tabela II: Features derivadas — definição exata e codificação (FeatureEngineering / DataPreprocessing)

Feature	Família	Definição	Valores / Codificação
MONTH	Temporal	Mês extraído de FL_DATE	Inteiro 1–12
DAY_OF_WEEK	Temporal	Dia da semana de FL_DATE	0=Segunda ... 6=Domingo
IS_WEEKEND	Temporal (binária)	1 se DAY_OF_WEEK $\geq$ 5	{0, 1}
SEASON	Temporal	Estação derivada do mês	1=Inverno (12,1,2), 2=Primavera (3–5), 3=Verão (6–8), 4=Outono (9–11)
IS_HOLIDAY_MONTH	Temporal (binária)	1 se mês $\in$ {7, 8, 12} (julho, agosto, dezembro)	{0, 1}
CRS_DEP_HOUR	Temporal	CRS_DEP_TIME // 100 (hora planeada de partida)	Inteiro 0–23
CRS_ARR_HOUR	Temporal	CRS_ARR_TIME // 100 (hora planeada de chegada)	Inteiro 0–23
TIME_OF_DAY	Temporal	Período do dia (pd.cut sobre CRS_DEP_HOUR)	0=Madrugada (0–5h), 1=Manhã (6–11h), 2=Tarde (12–18h), 3=Noite (19–23h)
ROUTE	Geográfica	Concatenação ORIGIN+“_”+DEST	Catagórica → <i>Label Encoding</i>
FLIGHT_TYPE	Operacional	Faixa de distância do voo (pd.cut sobre DISTANCE)	0=Curto ($\leq$ 500 mi), 1=Médio (501–1500), 2=Longo ($>$ 1500)
PLANNED_SPEED	Operacional	Velocidade planeada ($\approx$ DISTANCE/CRS_ELAPSED_TIME)	Contínua → <i>StandardScaler</i>
AVG_DELAY_PER_HOUR	Target encoding	Média de ARR_DELAY por CRS_DEP_HOUR, ajustada só no treino	Contínua → <i>StandardScaler</i>

simultaneamente disponível para a tarefa de regressão:

$$\text{DELAY_CLASS} = \begin{cases} 0 \text{ (On-time)} & \text{se } \text{ARR_DELAY} < 15 \\ 1 \text{ (Short)} & \text{se } 15 \leq \text{ARR_DELAY} \leq 30 \\ 2 \text{ (Long)} & \text{se } \text{ARR_DELAY} > 30 \end{cases} \quad (1)$$

Os limiares de 15 e 30 minutos derivam da prática operacional da aviação (o *threshold* de 15 min é o padrão do DOT para considerar um voo “atrasado”). Esta dupla formulação — classificação e regressão sobre o mesmo alvo — permite, mais tarde, contrastar a previsibilidade da *categoria* com a do *valor exato*.

F. Disciplina Anti-Leakage no Target Encoding

A divisão treino/teste ocorre cedo (no DataLoader: `test_size=0.2, random_state=42`), antes de qualquer engenharia de *features*. É esta ordem que torna o *target encoding* seguro: a função de engenharia é invocada com o alvo *apenas* para o treino — calculando o mapa hora → atraso médio (`groupby('CRS_DEP_HOUR')`) e retendo-o em memória — e, para o teste, é invocada *sem* o alvo, limitando-se a aplicar o mapa já aprendido. Horas do teste ausentes no treino recebem a média global do treino. Assim, nenhuma informação de atraso do conjunto de teste participa na construção de qualquer *feature*.

G. Codificação e Escalonamento

A transformação final dos dados distinguiu três tratamentos conforme a natureza de cada variável:

- **Label Encoding** para as variáveis categóricas em texto (ORIGIN, DEST, AIRLINE_CODE, ROUTE). Para garantir que nenhum código de aeroporto ou rota presente apenas no teste provoca uma falha em inferência, cada codificador é ajustado sobre a *união* dos valores de treino e teste — uma partilha de vocabulário categórico que não

envolve a variável alvo e, portanto, não constitui *leakage* de informação preditiva.

- **StandardScaler** (média 0, desvio-padrão 1) para as variáveis contínuas: CRS_DEP_TIME, CRS_ARR_TIME, CRS_ELAPSED_TIME, DISTANCE, CRS_DEP_HOUR, CRS_ARR_HOUR, PLANNED_SPEED e AVG_DELAY_PER_HOUR.
- **MinMaxScaler** (intervalo [0, 1]) para as restantes variáveis (categóricas já codificadas numericamente).

A separação entre dois escalonadores é uma decisão técnica deliberada: a estandardização é apropriada para grandezas contínuas com distribuições aproximadamente simétricas após transformação, enquanto a normalização *min-max* mantém os códigos categóricos num intervalo limitado e comparável. Os escalonadores (*StandardScaler* e *MinMaxScaler*) são **ajustados apenas no treino** e aplicados ao teste, e a sua serialização permite a posterior operacionalização (Fase 6). Este cuidado de escala é particularmente importante para o KNN, cujas distâncias Euclidianas seriam dominadas por qualquer dimensão de escala desproporcionada.

III. FASE 2: ANÁLISE EXPLORATÓRIA DE DADOS

A análise exploratória teve um objetivo metodológico preciso: determinar *a priori* que tipo de modelo o problema exige. Em concreto, procurou-se responder a uma pergunta — *existe estrutura linear que ligue as features ao atraso, ou o problema é intrinsecamente não-linear?* A resposta condiciona toda a Fase 4. Para tal recorreu-se a quatro instrumentos complementares: análise de distribuições, correlação linear (Pearson), importância não-linear por permutação, e redução de dimensionalidade (PCA, t-SNE e UMAP).

A. Distribuições e Relações Bivariadas

A primeira etapa mapeou a dispersão das variáveis e a sua interação com o atraso. As variáveis contínuas — como

o tempo de voo planeado (CRS_ELAPSED_TIME) e a distância (DISTANCE) — apresentam distribuições fortemente assimétricas à direita e ampla gama de valores (Fig. 1). Esta observação tem dupla consequência: justifica a estandardização aplicada na Fase 1 e, sobretudo, **condiciona a escolha dos testes estatísticos** da Fase 3 — uma distribuição alvo enviesada viola a premissa de normalidade e exige testes não-paramétricos. A análise dos atrasos por categoria, através de *boxplots* (Fig. 2), mostrou que a cauda de atrasos extremos se alarga significativamente em categorias específicas — meses de férias e voos noturnos — antecipando visualmente o sinal que os testes de hipóteses viriam a confirmar matematicamente.

B. Análise de Correlação Linear (Heatmap)

Para quantificar a força das relações lineares diretas entre as variáveis numéricas e o atraso à chegada (ARR_DELAY), construiu-se uma matriz de correlação de Pearson (Fig. 3). A leitura da última linha/coluna — a correlação de cada *feature* com o alvo — é reveladora: todos os coeficientes são **quase nulos**. A distância e a velocidade planeada exibem $r \approx 0,01$, os horários e tempos planeados $r \leq 0,08$, e mesmo a melhor variável, a AVG_DELAY_PER_HOUR obtida por *target encoding*, atinge apenas $r \approx 0,09$. Em contraste, o *heatmap* revela correlações elevadas *entre features* (e.g. CRS_DEP_TIME vs CRS_DEP_HOUR $\approx 1,00$; DISTANCE vs CRS_ELAPSED_TIME $\approx 0,97$), confirmando redundância esperada entre pares derivados da mesma grandeza.

Esta é a **primeira prova matemática** de que não existe relação linear simples que explique os atrasos: o atraso não cresce proporcionalmente com a distância nem com a hora exata de partida de forma isolada, resultando antes de uma combinação complexa e contextual de fatores. A consequência metodológica é direta — modelos lineares (e.g. regressão linear simples) estão votados ao fracasso, o que motiva o recurso a algoritmos não-lineares e baseados em árvores.

C. Importância das Variáveis (Permutation Importance)

Como a correlação linear se revelou insuficiente, recorreu-se a um método não-linear robusto: a importância por permutação. Treinou-se um *Random Forest Regressor* base e mediu-se, para cada coluna, o aumento do erro do modelo provocado pela aleatorização (*shuffling*) dos seus valores — uma *feature* é tanto mais importante quanto mais o seu embaralhamento degrada a previsão (Fig. 4).

O resultado mais informativo emerge do **contraste com a correlação de Pearson**. As variáveis contínuas de horário e rota — CRS_ARR_TIME, CRS_DEP_TIME, DISTANCE e PLANNED_SPEED — lideram a importância por permutação, apesar de exibirem correlação linear quase nula com o alvo. Esta aparente contradição é, na verdade, a **prova mais forte da natureza não-linear** do problema: estas variáveis carregam sinal preditivo genuíno, mas só acessível através das interações e fronteiras não-lineares que a *Random Forest* consegue modelar e que um simples coeficiente linear é incapaz de detetar. As *features* temporais categóricas (TIME_OF_DAY,

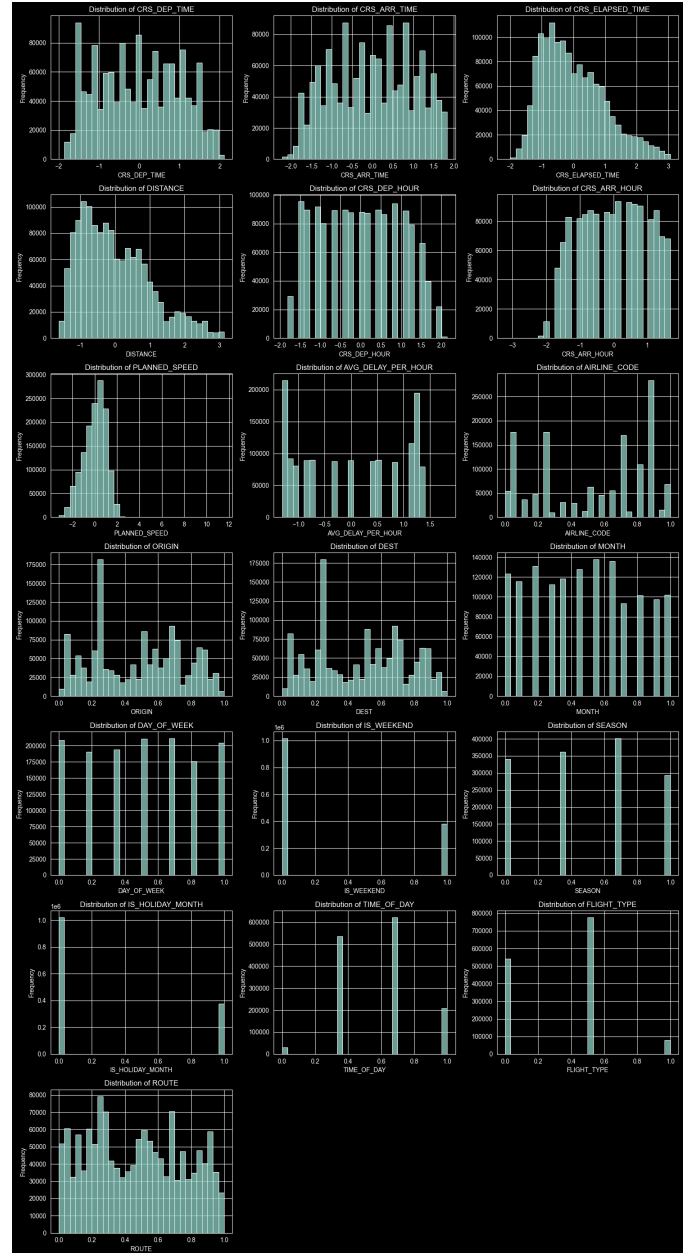


Figura 1: Distribuições das *features* numéricas e categóricas. As variáveis contínuas exibem forte assimetria à direita, justificando a estandardização e a escolha de testes não-paramétricos.

FLIGHT_TYPE), embora com importância marginal mais baixa neste teste *univariado* de permutação, revelar-se-ão determinantes para a *estrutura global* dos dados na análise de redução de dimensionalidade que se segue. Em conjunto, os dois resultados confirmam que nenhuma *feature* isolada explica o atraso de forma linear, justificando definitivamente o recurso a modelos não-lineares.

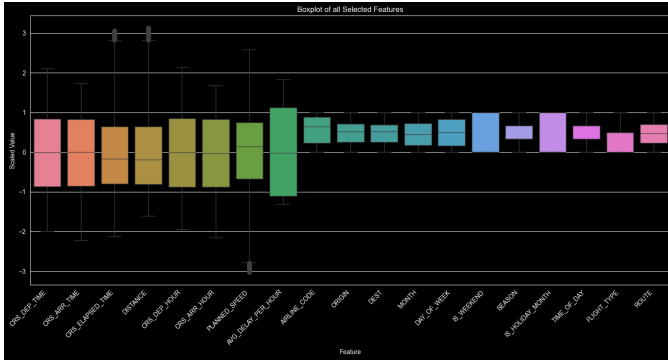


Figura 2: *Boxplots* das *features* selecionadas. A amplitude das caudas confirma a presença de atrasos extremos concentrados em categorias operacionais e temporais específicas.



Figura 3: Matriz de correlação de Pearson entre as variáveis numéricas e `ARR_DELAY`. Os coeficientes residuais com a variável alvo provam a ausência de relação linear direta.

D. Redução de Dimensionalidade: Linear vs. Não-Linear

Para investigar a estrutura global dos dados aplicaram-se três técnicas complementares — um método linear (PCA) e dois não-lineares (t-SNE e UMAP) — todas sobre as *features* estandardizadas. A Tabela III resume a configuração exata de cada algoritmo. A lógica experimental é uma prova por contraste: se um método linear falha mas um não-linear sucede, a não-linearidade do problema fica demonstrada.

A projeção 2D do **PCA** (Fig. 5) — que procura as combinações lineares de máxima variância — revelou-se incapaz de separar as classes *On-time* e *Delayed*, produzindo uma nuvem densa e totalmente sobreposta (com um padrão reticulado induzido pelas *features* discretas). As duas primeiras componentes capturam uma fração modesta da variância total, e nenhuma direção linear isola os voos atrasados. Este resultado confirma matematicamente que a estrutura do problema não é linearmente separável.

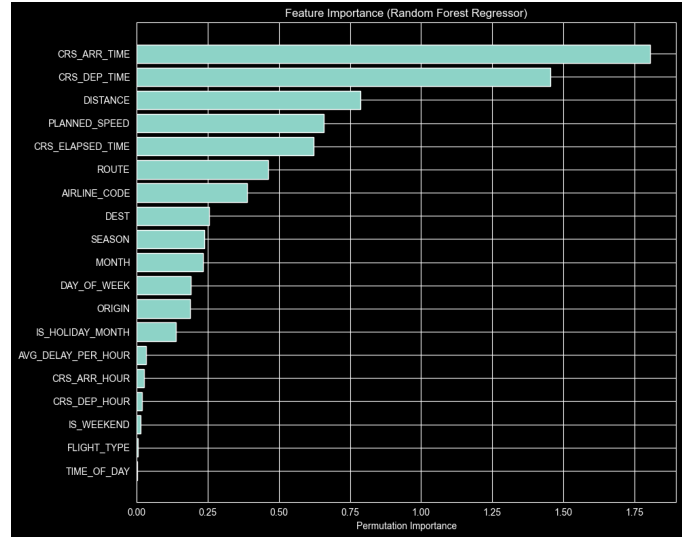


Figura 4: Importância por permutação (*Random Forest Regressor* base). As *features* contextuais engenheiradas superam as variáveis operacionais brutas, validando a fase de *Feature Engineering*.

Tabela III: Hiperparâmetros das técnicas de redução de dimensionalidade

Algoritmo	Parâmetros	Amostra
PCA	<code>n_components=2</code>	Conjunto completo
t-SNE	<code>perplexity=30,</code> <code>random_state=42</code>	5.000 ($O(N^2)$)
UMAP	<code>n_neighbors=15,</code> <code>min_dist=0.1,</code> <code>random_state=42</code>	Conjunto completo

O **t-SNE** (Fig. 6), aplicado a uma sub-amostra controlada de 5.000 pontos devido à sua complexidade $O(N^2)$, começou a revelar agrupamentos fragmentados com base nas *features* operacionais. Preserva bem a estrutura local (pontos vizinhos no espaço original mantêm-se vizinhos) mas, por construção, sacrifica a noção das distâncias globais entre *clusters*, pelo que serve de indício, não de prova.

O **UMAP** (Fig. 7) produziu agrupamentos distintos e bem separados. Colorindo as projeções pela paleta *viridis* (Fig. 8), foi possível atribuir cada eixo de variação a uma *feature* concreta:

- **Espaço entre *clusters*:** dominado por `FLIGHT_TYPE` — voos de curto, médio e longo curso formam “ilhas” isoladas, correspondentes a perfis operacionais distintos.
- **Subdivisão dos *clusters*:** `IS_HOLIDAY_MONTH` separa cada grupo em sub-*clusters* de férias vs. não-férias.
- **Gradiente interno:** `TIME_OF_DAY` organiza os pontos dentro de cada ilha (manhã, tarde, noite).

O contraste entre a falha do PCA e o sucesso do UMAP indica que o conjunto de dados é **altamente estruturado, mas estritamente não-linear**. Importa notar que a separação observada no UMAP é, em parte, induzida pelas próprias *features*

categóricas engenheiradas; o seu valor está em demonstrar que essas *features* carregam estrutura coerente, justificando o uso de modelos não-lineares e baseados em árvores nas fases seguintes — e antecipando o desfecho central do projeto: nenhuma destas estruturas pré-partida chega para prever o atraso *ao minuto*.

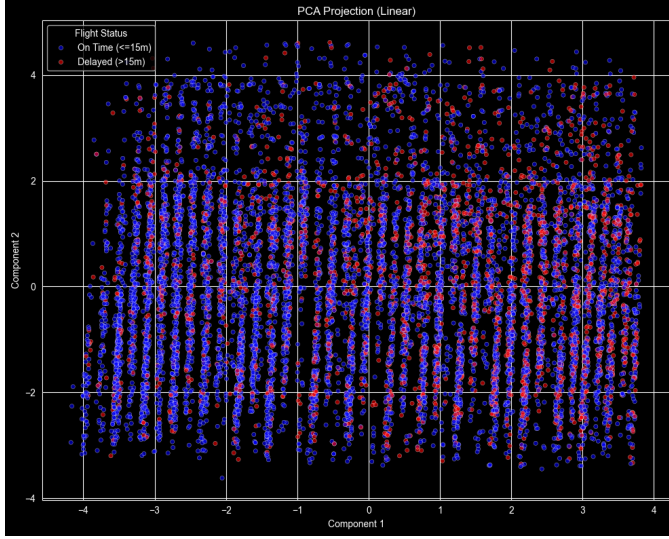


Figura 5: Projeção PCA (linear). A sobreposição completa das classes *On-time* e *Delayed* prova a inadequação de modelos lineares.

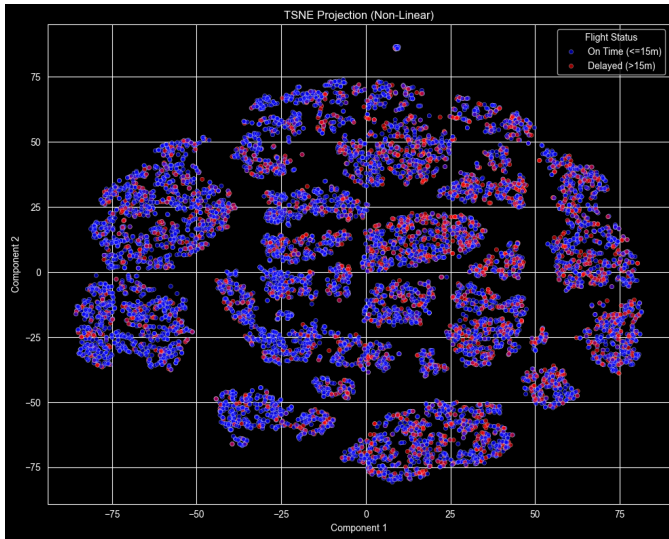


Figura 6: Projeção t-SNE (não-linear) sobre sub-amostra. Surgem agrupamentos locais fragmentados; a estrutura global não é preservada.

IV. FASE 3: TESTES DE HIPÓTESES E SELEÇÃO DE MODELOS

A. Formulação e Metodologia

Onde a EDA sugeriu visualmente, os testes de hipóteses provam formalmente. O objetivo foi validar, com rigor esta-

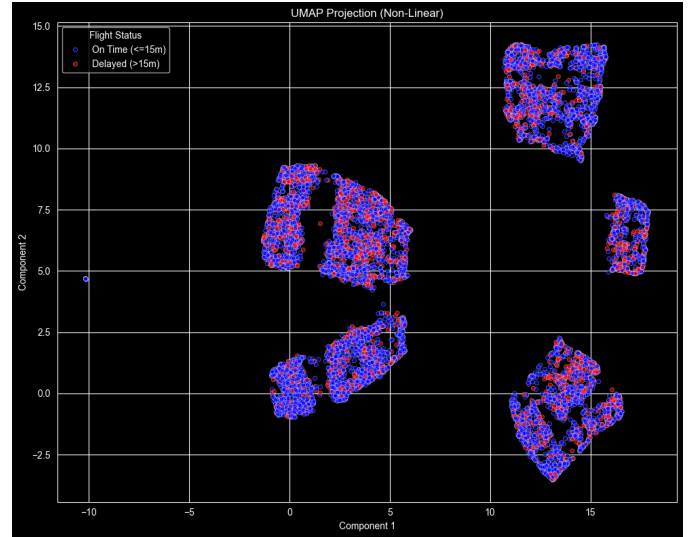


Figura 7: Projeção UMAP (não-linear). Agrupamentos distintos e separados, estruturados pelas *features* operacionais e temporais.

tístico, que cada *feature* engenheirada tem relação significativa com o atraso — confirmando que não são ruído antes de as confiar a um modelo. Para cada variável formulou-se o par de hipóteses padrão, com nível de significância $\alpha = 0,05$:

- H_0 : a distribuição de *ARR_DELAY* é igual entre os grupos definidos pela *feature*;
- H_1 : pelo menos um grupo difere.

A escolha dos testes foi cuidadosamente adaptada à natureza empírica dos dados. Como a EDA demonstrou, a variável alvo é fortemente enviesada à direita, violando a premissa de normalidade. Por essa razão, a **ANOVA paramétrica foi deliberadamente descartada** — produziria valores-*p* pouco fiáveis — em favor de duas alternativas:

- **Teste t de Welch** (*equal_var=False*): para as variáveis binárias (*IS_WEEKEND*, *IS_HOLIDAY_MONTH*), comparando a média de atraso entre dois grupos. Optou-se pela variante de Welch, e não pelo t de Student clássico, por *não* assumir igualdade de variâncias entre os grupos — premissa irrealista dado o desequilíbrio dimensional e a heterocedasticidade observada.
- **Teste de Kruskal-Wallis**: alternativa não-paramétrica à ANOVA (baseada em ordenações, não em médias), para as variáveis multiclasse (*TIME_OF_DAY*, *SEASON*, *FLIGHT_TYPE*, *DAY_OF_WEEK*, *MONTH*). Por operar sobre *ranks*, é robusto à assimetria e aos *outliers* da cauda direita.

A validação das *features* numéricas contínuas, por sua vez, usou a correlação de **Pearson** com o respetivo teste de significância.

B. Resultados

A Tabela IV consolida os resultados. **Todas** as variáveis categóricas engenheiradas apresentaram relação altamente significativa com o atraso, com valores-*p* ordens de grandeza

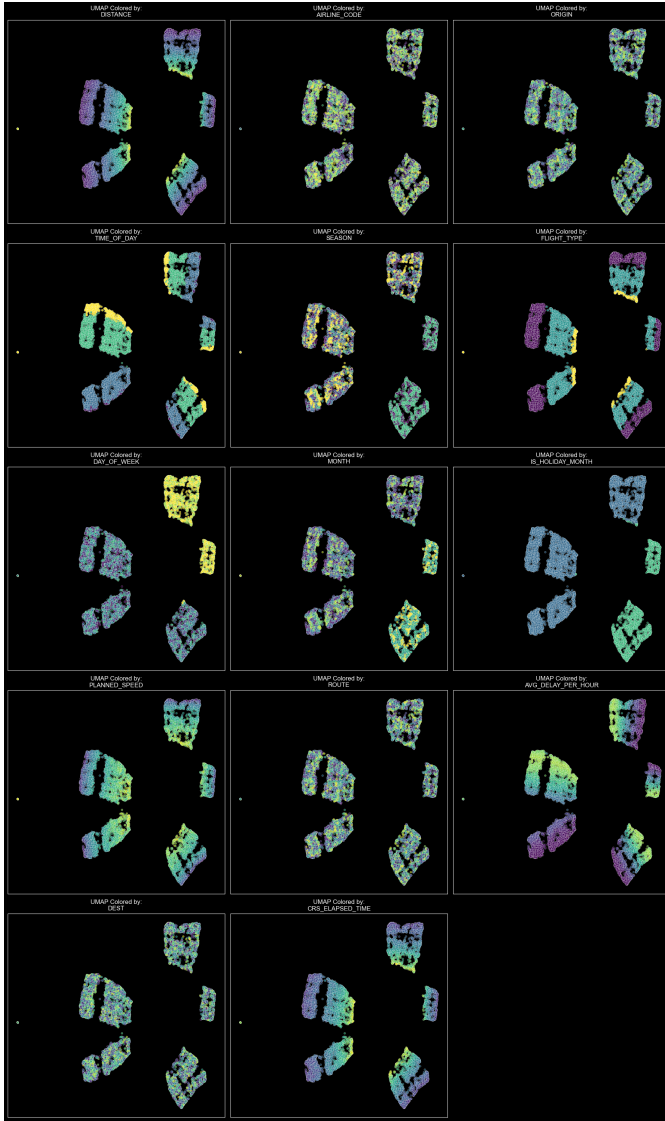


Figura 8: UMAP colorido por 14 *features* distintas (paleta *viridis*). Cada *feature* explica um eixo de variação: `FLIGHT_TYPE` separa ilhas, `IS_HOLIDAY_MONTH` subdivide, `TIME_OF_DAY` cria gradientes internos.

abaixo de α . Nos testes de Welch, tanto o fim de semana ($p \approx 1,35 \times 10^{-15}$) como — de forma extrema — o mês de férias ($p \approx 7,38 \times 10^{-305}$) afetam significativamente o atraso. Os testes de Kruskal-Wallis para as variáveis multiclassee devolveram $p < 0,0001$ (virtualmente zero). Rejeita-se assim, firmemente, H_0 para a totalidade das *features*.

Quanto às variáveis numéricas, a correlação de Pearson confirma o quadro da EDA: o *Target Encoding* foi a engenharia mais bem-sucedida — `AVG_DELAY_PER_HOUR` apresenta a correlação mais forte com o alvo ($r \approx 0,09$) — enquanto a distância e a velocidade planeada exibem correlações residuais ($r \approx 0,01$). Há aqui uma síntese importante: as *features* são estatisticamente significativas (têm sinal real) mas linearmente fracas (o sinal não é linear). Significância estatística e força

Tabela IV: Resultados dos testes de hipóteses ($\alpha = 0,05$)

Variável	Teste	valor- p	H_0
IS_WEEKEND	Welch t	$1,35 \times 10^{-15}$	× Rejeita
IS_HOLIDAY_MONTH	Welch t	$7,38 \times 10^{-305}$	× Rejeita
TIME_OF_DAY	Kruskal-Wallis	$< 10^{-4}$	Rejeita
SEASON	Kruskal-Wallis	$< 10^{-4}$	Rejeita
FLIGHT_TYPE	Kruskal-Wallis	$< 10^{-4}$	Rejeita
DAY_OF_WEEK	Kruskal-Wallis	$< 10^{-4}$	Rejeita
MONTH	Kruskal-Wallis	$< 10^{-4}$	Rejeita

linear são dimensões distintas, e é precisamente essa distinção que sustenta a opção por modelos não-lineares.

C. Estratégia de Seleção de Modelos e Avaliação

Concluída a preparação e a validação estatística, a evidência acumulada (falha do PCA, importância por permutação, correlações lineares nulas mas significância estatística máxima) aponta de forma unânime para um problema **não-linear**. A estratégia de modelação foi desenhada em conformidade:

- **KNN *from scratch*** como *baseline* não-paramétrico, tirando partido do espaço estandardizado;
- **modelos baseados em árvores** (Árvore de Decisão, Random Forest, Gradient Boosting), naturalmente aptos a capturar interações não-lineares;
- **redes neurais** (MLP) para dados tabulares; e
- **clustering** de aeroportos para análise não supervisionada.

A metodologia de avaliação privilegia métricas robustas ao desequilíbrio de classes — Precisão, *Recall*, F1 por classe e, sobretudo, **Macro F1** (média não ponderada que penaliza igualmente os erros em todas as classes) — com a matriz de confusão como instrumento analítico central para distinguir falsos positivos de falsos negativos entre severidades. O tratamento do desequilíbrio é feito em duas camadas: um balanceamento binário global (50/50 pontual/atrasado) já aplicado no carregamento (Fase 1) e, sobre o remanescente desequilíbrio *triclasse* (73/10/17), o uso de `class_weight='balanced'` em cada modelo e, em experiências dedicadas (Exp. 2), *undersampling* adicional até à classe minoritária — com validação cruzada na otimização de hiperparâmetros. Esta articulação entre preparação e modelação é o fio condutor que liga a Fase 1 às Fases 4–6.

V. FASE 4: CONSTRUÇÃO DE MODELOS

A. K-Nearest Neighbors — Implementação de Raiz (*Knn.py*)

O requisito de implementação *from scratch* foi cumprido com exclusividade: a classe `Knn` usa apenas `numpy` e `collections.Counter`, sem qualquer chamada a bibliotecas de ML. Isto torna o KNN o único modelo do projeto em que cada linha de código de inferência é visível e auditável.

1) *Algoritmo e Complexidade*: O método `fit` simplesmente memoriza o conjunto de treino como arrays NumPy — o KNN é um *lazy learner* e não tem fase de otimização.

Na inferência, para cada ponto de teste $\mathbf{x}$, a distância Euclidiana é calculada **vetorialmente** contra todo o treino via `np.linalg.norm`:

$$d(\mathbf{x}, \mathbf{x}_i) = \|\mathbf{x} - \mathbf{x}_i\|_2 = \sqrt{\sum_{j=1}^d (x_j - x_{i,j})^2} \quad (2)$$

Os k índices mais próximos são selecionados por `np.argsort(distances)[:k]` e a classe é determinada por votação maioritária via `Counter.most_common(1)`:

$$\hat{y} = \arg \max_c \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}[y_i = c] \quad (3)$$

A complexidade de inferência por ponto é $O(N \cdot d)$ com N = tamanho do treino e d = dimensionalidade, tornando o custo total $O(N_{\text{test}} \cdot N_{\text{train}} \cdot d)$. Com $N_{\text{train}} = 200\,000$ e $d = 22$ (features numéricas do dataset preprocessado), a inferência no conjunto completo de 50.000 pontos de teste levaria $\approx 2,2$ mil milhões de operações por ponto — impraticável sem aceleração.

2) *Configuração Exacta e Dados de Entrada*: O modelo é treinado com $k = 5$ sobre o conjunto preprocessado e escalonado (`data_loader_preprocessed.pkl`). As *features* categóricas (codificadas com *LabelEncoder* + *MinMaxScaler*) e as numéricas (*StandardScaler*) estão todas no mesmo espaço escalonado — **condição indispensável** para que a distância Euclidiana seja equitativa entre dimensões. A variável alvo é a classe triclasse, aplicada pela mesma função de categorização usada em todos os outros modelos: $ARR_DELAY < 15 \rightarrow 0$; $[15, 30] \rightarrow 1$; $> 30 \rightarrow 2$.

A avaliação foi restrita a **5.000 exemplos de teste** (`X_test_scaled[:5000]`) — os primeiros 5.000 da divisão original — imposta pela inviabilidade computacional da pesquisa exaustiva. Esta granularidade limita a robustez estatística das estimativas, em particular para as classes minoritárias.

Tabela V: KNN — Desempenho ($k = 5$, $n_{\text{train}} = 200\,000$, $n_{\text{test}} = 5\,000$)

Métrica	Valor	Nota
Exatidão	$\approx 60\%$	Sub-amostra 5k
F1 <i>On-time</i>	≈ 0.72	Classe maioritária
F1 <i>Short delay</i>	≈ 0.14	Classe mais difícil
F1 <i>Long delay</i>	≈ 0.25	
Macro F1	≈ 0.37	

3) *Análise dos Resultados*: O KNN estabelece o *baseline* não-paramétrico. O Macro F1 ≈ 0.37 demonstra que a distância Euclidiana no espaço escalonado captura sinal relevante: os vizinhos mais próximos de um voo são, em média, melhores preditores do que um classificador aleatório ou um preditor de classe maioritária. Porém, o F1 de *Short* = 0.14 documenta o problema estrutural da classe intermédia — a sua janela de 15 min é tão estreita que muitos vizinhos de um voo *Short* verdadeiro são voos *On-time* ou *Long*, levando à votação maioritária errada.

A desvantagem operacional é irrefutável: sem estruturas de índice (KD-Tree, Ball-Tree), o KNN é um algoritmo de produção inviável nesta escala. A sua utilidade é estritamente de *baseline* e de prova de conceito da implementação.

B. Aprendizagem Supervisionada Clássica (*SupervisedModels.py*)

Os dois modelos supervisionados clássicos foram implementados na classe *SupervisedModels*, que os instancia num dicionário e itera sobre ambos com a mesma interface de treino e avaliação. Os dados de entrada são os do checkpoint preprocessado, subamostrados a **100.000 exemplos de treino** e avaliados no conjunto completo de **50.000 de teste**. A subamostragem de treino é necessária pela velocidade de ajuste, em particular da Regressão Logística com `max_iter=3000`.

1) *Árvore de Decisão* (*DecisionTreeClassifier*): A *Decision Tree* particiona o espaço de *features* por limiares binários recursivos, maximizando a redução de impureza de Gini em cada nó. Parâmetros exactos: `random_state=42`, `class_weight='balanced'`; **sem restrição de profundidade** (`max_depth=None`), o que significa que a árvore cresce até à pureza total nos nós folha — comportamento com *overfitting* expectável sobre o treino, mas com capacidade máxima de capturar interações não-lineares complexas.

O `class_weight='balanced'` ajusta os pesos de cada classe inversamente proporcional à sua frequência no treino: $w_c = \frac{N}{k \cdot N_c}$, onde N é o número total de exemplos, k o número de classes e N_c a frequência da classe c . Isto compensa o desequilíbrio 73/10/17 sem remover dados.

2) *Regressão Logística* (*LogisticRegression*): Estima probabilidades de classe por *softmax* multiclasse sobre uma combinação linear das *features*. Parâmetros: `max_iter=3000` (necessário para convergência no espaço de alta dimensão e desequilíbrio), `random_state=42`, `class_weight='balanced'`, `n_jobs=-1` (paralelização total). O solver padrão do sklearn (*lbfgs*) é usado, adequado para problemas multiclasse com dimensões moderadas.

Tabela VI: Modelos Supervisionados — Resultados ($n_{\text{treino}} = 100\,000$, $n_{\text{teste}} = 50\,000$)

Modelo	Acc.	F1 _{ON}	F1 _{SH}	F1 _{LG}	Macro F1
Árvore de Decisão	$\approx 55\%$	≈ 0.67	≈ 0.17	≈ 0.26	≈ 0.37
Reg. Logística	$\approx 57\%$	≈ 0.70	≈ 0.15	≈ 0.23	≈ 0.36

3) *Análise dos Resultados*: Ambos os modelos atingem Macro F1 semelhante (≈ 0.36 – 0.37), mas por razões distintas. A Árvore de Decisão — sem profundidade limitada — é propensa a *overfit*: memoriza padrões locais do treino mas generaliza mal para voos *Long* pouco representados (F1 = 0.26). A Regressão Logística, sendo um modelo linear, só captura fronteiras de decisão planas no espaço das *features*; a sua exatidão ligeiramente superior (57% vs 55%) reflete maior robustez à variância, não maior capacidade expressiva.

O padrão dominante nas matrizes de confusão (Fig. 9) é a **confusão massiva de Short com On-time**: os modelos lineares/raso não conseguem traçar uma fronteira de decisão

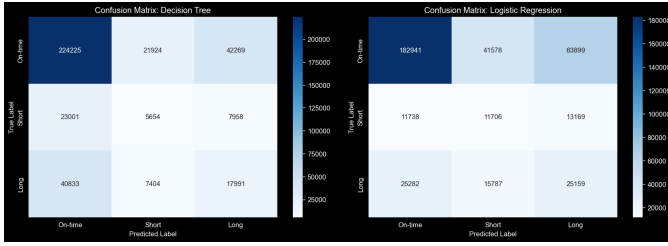


Figura 9: Matrizes de confusão — Árvore de Decisão (esq.) e Regressão Logística (dir.). O erro dominante é a confusão de *Short* com *On-time*: ambos os modelos subclassificam a classe 1 por insuficiência de sinal na janela de 15 min.

coerente dentro da faixa estreita 15–30 min. Este resultado antecipa o que se observará em todos os modelos subsequentes e motiva a formulação hierárquica.

C. Modelos de Ensemble (*Ensemble.py*)

Os dois *ensembles* foram implementados na classe *Ensemble*, que suporta subsampling opcional e balanceamento de treino. Foram executadas duas experiências com os mesmos 200.000 exemplos de treino e 50.000 de teste: **Exp. 1** — distribuição original sem reamostragem; **Exp. 2** — *undersampling* triclasse até à classe minoritária (*Short*, $\approx 10\%$) — produzindo um treino perfeitamente balanceado a 33%/33%/33% via selecção aleatória com `np.random.default_rng`.

1) *Random Forest* — *Bagging* (*RandomForestClassifier*): O *Random Forest* [4] constrói um conjunto de `n_estimators=100` árvores, cada uma treinada sobre uma amostra *bootstrap* com reposição do conjunto de treino e avaliando apenas um subconjunto aleatório de *features* em cada nó. Esta dupla aleatoriedade (amostragem de dados + amostragem de *features*) descorrela as árvores individualmente e reduz a variância do conjunto. Parâmetros exactos usados na classe *Ensemble*: `n_estimators=100`, `max_depth=None` (árvores crescem até pureza), `random_state=42`, `n_jobs=-1`. A inexistência de `class_weight` nesta configuração significa que o desequilíbrio de classes (73/10/17) é tratado *apenas* pela reamostragem na Exp. 2 — na Exp. 1, as árvores otimizam sem penalização de classe.

2) *HistGradientBoosting* — *Boosting* (*HistGradientBoostingClassifier*): O *HistGradientBoosting* [5] treina árvores *sequeencialmente*: cada nova árvore ajusta os *pseudo-resíduos* (gradiente da *loss* em relação à predição acumulada) das anteriores. A variante de histograma agrupa os valores contínuos de cada *feature* em *bins* discretos antes do treino, reduzindo a complexidade de busca de limiares de $O(N)$ para $O(B)$ por nó (onde $B \ll N$ é o número de *bins*), tornando-o 5–10× mais rápido do que o GBM clássico. Parâmetros: `max_iter=200`, `learning_rate=0.1`, `max_depth=None`, `random_state=42`.

Tabela VII: Ensemble — Configuração e Resultados Comparativos

Exp.	Mod.	Dados	Acc.	F1 _{ON}	F1 _{SH}	F1 _{LG}	Macro F1
1	RF	Original	$\approx 63\%$	≈ 0.76	≈ 0.21	≈ 0.10	≈ 0.35
	HGB	Original	$\approx 65\%$	≈ 0.77	≈ 0.24	≈ 0.13	≈ 0.38
2	RF	Balanceado	$\approx 49\%$	≈ 0.58	≈ 0.23	≈ 0.28	≈ 0.36
	HGB	Balanceado	$\approx 51\%$	≈ 0.60	≈ 0.25	≈ 0.29	≈ 0.38

3) *Análise dos Resultados e Trade-offs*: A Exp. 1 confirma que o HGB supera o RF em exatidão (+2 p.p.) e em Macro F1 (0.38 vs 0.35), beneficiando da sequencialidade do ajuste de resíduos: cada iteração foca-se precisamente nos voos que as iterações anteriores erraram. O RF, por construção paralela e independente das árvores, não tem este mecanismo de correção dirigida.

O resultado mais instrutivo é o *trade-off* entre Exp. 1 e Exp. 2. O balanceamento triclasse reduz drasticamente a exatidão global (−14 p.p. para o RF, −14 p.p. para o HGB) mas **melhora o F1 de Long delay** em +0.15–0.17 p.p. — a classe com menor representação beneficia mais da eliminação do viés maioritário. O Macro F1, por ser não-ponderado, permanece quase estável (0.35–0.38), confirmando que o ganho nas classes minoritárias compensa a perda na maioritária. Esta é a evidência empírica que motiva o uso do Macro F1 como métrica primária: exatidão global oculta o real desempenho em classes operacionalmente relevantes.

D. MLP (*DeepLearning.py*)

1) *Justificação da arquitetura e Decisões de Design*: O dataset é estritamente tabular: cada voo é uma linha independente descrita por um vetor de comprimento fixo. Não existe estrutura espacial (CNN) nem sequência temporal por amostra (RNN/LSTM) — as *features* temporais já estão engenheiradas como atributos estáticos. O *Multi-Layer Perceptron* (MLP) é a arquitetura canónica para dados tabulares [2]: consegue aprender fronteiras de decisão arbitrariamente não-lineares e é implementado com `tf.keras.Sequential`.

A arquitetura base (Tabela VIII) usa três camadas ocultas de tamanhos decrescentes (128 → 64 → 32), cada uma seguida de *BatchNormalization* e *Dropout*. As dimensões decrescentes induzem uma hierarquia de representações do geral para o específico. O *BatchNorm* acelera a convergência ao normalizar as ativações de cada *mini-batch*, reduzindo o desvio interno de covariáveis. O *Dropout* com taxa 0.3 nas primeiras duas camadas e 0.2 na última atua como regularizador ao desativar aleatoriamente neurónios durante o treino, impedindo a co-adaptação excessiva.

Parâmetros de treino comuns a todas as variantes: Otimizador Adam ($\text{lr} = 10^{-3}$); 80 épocas máximas; *batch size* 512; divisão interna de validação 20% (`train_test_split` estratificado por classe); `ReduceLROnPlateau`(`monitor=val_loss`, `factor=0.5`, `patience=5`). A *class_weight* é calculada automaticamente pelo `sklearn`

Tabela VIII: arquitetura MLP Partilhada — Camada por Camada

Camada	Configuração	Papel
Dense 128	ReLU + BatchNorm + Dropout(0.3)	Representação primária
Dense 64	ReLU + BatchNorm + Dropout(0.3)	Compressão
Dense 32	ReLU + BatchNorm + Dropout(0.2)	Abstracção final
Dense 3	Softmax / bias = $\log p_0$	Classificação 3 classes
Dense 2	Softmax / bias = $\log p_0$	Classificação binária
Dense 1	Linear	Regressão em minutos

(`compute_class_weight('balanced', ...)`) para a classificação sem *undersampling*.

Inicialização do *bias* de saída com $\log(\text{priors})$: A saída *softmax* converge, no início do treino, para os *priors* das classes se o *bias* for inicializado a $\log(p_c)$ (onde p_c é a proporção da classe c no treino). Isto dá à rede um ponto de partida que já corresponde à distribuição marginal — o otimizador não desperdiça épocas iniciais a aprender o *prior* e começa logo a ajustar as fronteiras de decisão.

2) **Bug Estrutural: Colapso para Preditor Maioritário:** O primeiro treino produziu 73% de exatidão com *recall* 1.00/0.00/0.00 — o modelo prediz *On-time* para **todos** os voos (preditor degenerado). A causa raiz é uma interação subtil entre quatro decisões simultâneas do Keras:

- 1) `class_weight='balanced'` re-pondera a *loss* de treino, tornando os exemplos raros mais influentes durante a retropropagação.
- 2) A *val_loss* (*cross-entropy* de validação) é calculada **sem ponderação** — é a *loss* bruta sobre a distribuição real 73/10/17.
- 3) Com esta distribuição, a *cross-entropy* não ponderada é *minimizada* pelo preditor constante-maioritário: prever *On-time* com probabilidade 1 dá $-\log(0.73) \approx 0.31$, abaixo de qualquer predição incerta.
- 4) A inicialização de *bias* com $\log(\text{priors})$ faz com que as primeiras épocas — antes de qualquer aprendizagem útil — já produzam um softmax próximo dos *priors*, gerando a **menor *val_loss*** de toda a execução. `EarlyStopping.restore_best_weights=True` reverte então o modelo a esse estado inicial degenerado, descartando todo o progresso subsequente.

Este *bug* é particularmente insidioso porque a exatidão de 73% parece razoável — só a inspecção do *recall* por classe revela o colapso.

3) **Correção: Callback `MacroF1Monitor`:** A solução substitui o `EarlyStopping(val_loss)` por um *callback* personalizado que monitoriza o **Macro F1 de validação**. O Macro F1 é imune ao colapso maioritário por construção: um preditor constante que classifica tudo como *On-time* obtém $F1=1$ nessa classe, $F1=0$ nas outras duas, e portanto $\text{Macro F1} \approx 0.33$ — nunca o máximo. O *callback* guarda os pesos

da melhor época em memória (`get_weights`) e restaura-os no fim (`set_weights`), replicando o comportamento de `restore_best_weights=True` mas para a métrica correcta. A condição de melhoria exige um aumento mínimo de 10^{-4} para evitar oscilações numéricas:

Listing 1: `MacroF1Monitor` (DeepLearning.py)

```
class MacroF1Monitor(Callback):
    def on_epoch_end(self, epoch, logs=None):
        preds = self.model.predict(
            self.X_val, verbose=0).argmax(axis=1)
        macro_f1 = f1_score(self.y_val, preds,
            average='macro', zero_division=0)
        if macro_f1 > self.best_f1 + 1e-4:
            self.best_f1 = macro_f1
            self.best_weights = self.model.get_weights()
            self.wait = 0
        else:
            self.wait += 1
            if self.wait >= self.patience:
                self.model.stop_training = True
    def on_train_end(self, logs=None):
        self.model.set_weights(self.best_weights)
```

Esta **única** alteração — sem mudar arquitetura, otimizador, dados ou pesos de classe — fez o Macro F1 passar de 0.28 (degenerado) para **0.42**.

4) **MLP Plano — Classificação (Exp. 1, 1b, 2):** Três variantes foram executadas sobre os mesmos 200k/50k exemplos (Tabela IX):

Tabela IX: MLP Plano — Três Variantes de Classificação

Exp.	Alteração	Acc.	F1 _{ON}	F1 _{SH}	F1 _{LG}	Macro F1
Exp. 1	MacroF1Mon.	70.0%	0.84	0.25	0.17	0.42
Exp. 1b	No BatchNorm + pesos {0 : 1, 1 : 5, 2 : 2.5}	73.0%	0.84	0.00	0.00	0.28 [†]
Exp. 2	<i>Undersampling</i> 3cl	26.5%	0.37	0.19	0.05	0.21

[†]Colapso maioritário — recall 1.00/0.00/0.00.

A **Exp. 1b** foi um *stress-test* de sobre-correcção: removeu-se o BatchNorm e usaram-se pesos manuais {0 : 1.0, 1 : 5.0, 2 : 2.5} em vez dos calculados pelo sklearn. O raciocínio era que forçar um peso maior na classe minoritária poderia resolver o colapso — mas o resultado foi o oposto: o modelo re-colapsou (Macro F1 = 0.28). A explicação é que as três intervenções simultâneas (remoção de BatchNorm, pesos mais extremos, retenção do mesmo callback) empurram a optimização de forma inconsistente e desestabilizam o gradiente. A Exp. 1b documenta que `MacroF1Monitor` é a intervenção **necessária e suficiente** — intervenções adicionais pioram.

A **Exp. 2** substituiu os pesos de classe pelo *undersampling* triclasse até à classe minoritária (Short, $\approx 10\%$), reduzindo o treino efectivo em $\approx 3\times$. A convergência prematura num conjunto menor degradou o desempenho (Macro F1 = 0.21), confirmando que a penalização via pesos é preferível à remoção de dados.

5) **MLP de Regressão Plana (Exp. 3):** O *backbone* idêntico, com saída **linear** (Dense 1) e *loss* MSE, foi treinado para prever `ARR_DELAY` em minutos directamente sobre o conjunto completo. O resultado é $R^2 = -28.04$ — dramaticamente pior do que um preditor de média. O mecanismo é preciso: o MSE

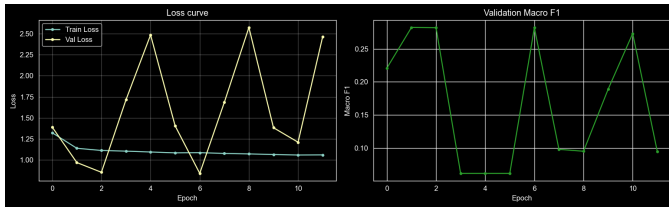


Figura 10: MLP Exp. 1 — curva de *loss* (esq.) e Macro F1 de validação (dir.) por época. A instabilidade da *val_loss* prova que não pode ser critério de paragem; o *MacroF1Monitor* termina na época de Macro F1 máximo (paciência 10 sem melhoria de $> 10^{-4}$).

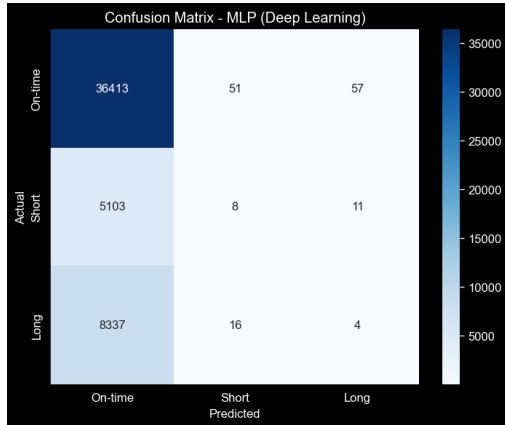


Figura 11: Matriz de confusão — MLP Plano Exp. 1. O padrão dominante é a classificação de voos *Short* e *Long* como *On-time*: o modelo aprendeu a fronteira de atraso relevante mas não consegue distinguir a sua magnitude com as *features* pré-partida disponíveis.

é minimizado por prever a média do alvo; com $\approx 73\%$ dos exemplos com $ARR_DELAY \approx 0$, a média global é próxima de zero, e o MSE de um preditor constante nulo sobre o conjunto completo é muito baixo. O regressor converge exactamente para este comportamento degenerado, e o R^2 negativo certifica que a rede é *pior* do que a média amostral. A variância residual é dominada por eventos do dia operacional ausentes do dataset: meteorologia em tempo real, atrasos em cascata por aeronave e congestionamento do espaço aéreo.

6) *MLP Hierárquico de 2 Estágios* (*HierarchicalDeepLearning.py*): Para decompor o problema em duas subtarefas especializadas, foi implementada uma cascata de dois estágios na classe *HierarchicalDeepLearning*.

Estágio 1 — Classificador Binário: arquitetura MLP partilhada com cabeça *softmax* de 2 saídas (*On-time* vs *Delayed*, ≥ 15 min). Treinado nos 200k exemplos completos com *class_weight='balanced'* computado pelo *sklearn* e *MacroF1Monitor* (*patience=10*). A divisão interna de validação é **estratificada por rótulo binário** (*stratify=y_train_bin*) para garantir representação equilibrada em ambas as partições.

Estágio 2 — Regressor de Minutos: Mesmo *backbone* com saída linear. Treinado **exclusivamente** nos voos atrasados do treino ($ARR_DELAY \geq 15$): a máscara *y_train_bin == 1* selecciona $\approx 100k$ voos com atraso, eliminando o ruído introduzido pelos 100k voos pontuais (alvo = 0). Para o Estágio 2, usa-se **EarlyStopping(val_loss)** — o problema de colapso maioritário não existe na regressão, onde a *val_loss* (MSE) é uma métrica de minimização directamente alinhada com o objetivo.

Inferência e composto triclasse: Para cada voo de teste, o Estágio 1 produz p_{delayed} ; se $p_{\text{delayed}} \geq 0.5$ (threshold fixo), o Estágio 2 prevê os minutos; caso contrário, prediz-se 0 min. O composto triclasse é obtido por *binning* da saída do Estágio 2: minutos $< 30 \rightarrow \text{Short}$; $\geq 30 \rightarrow \text{Long}$.

Tabela X: Hierárquico 2 Estágios — Métricas por Componente

Componente	Métrica	Valor	Âmbito
Estágio 1 (binário)	Exatidão	68.1%	50k teste
	F1 <i>Delayed</i>	0.475	
	Macro F1	0.623	
Estágio 2 (reg.)	RMSE	93.14 min	Subconj. atrasados
	MAE	45.73 min	
	R^2	-0.009	
Composto E2E	Exatidão (3cl.)	57.2%	50k teste
	Macro F1 (3cl.)	0.360	
	MAE ponta-a-ponta	45.7 min	

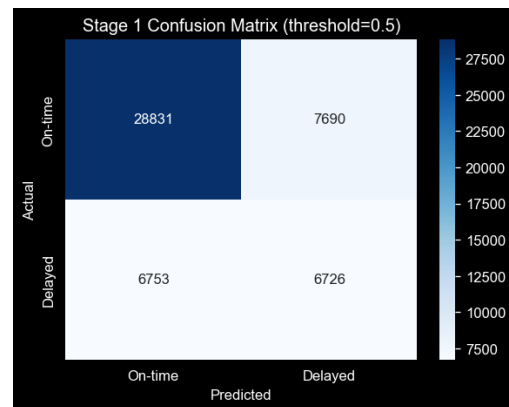


Figura 12: Matriz de confusão — Estágio 1 binário (threshold = 0.5). Macro F1 = 0.623: melhor resultado binário do projecto. O ganho face ao MLP triclasse (0.42) explica-se pela menor assimetria 73/27 vs 73/10/17 e pela concentração de capacidade numa única fronteira.

O Estágio 2 atinge $R^2 \approx -0.009$ mesmo treinado *apenas* em voos atrasados — quase igual a um preditor de média. Isto demonstra que o problema de regressão ao minuto é irreduzível com as *features* disponíveis, independentemente de filtrar os voos pontuais. O Estágio 1 binário, porém, é o melhor classificador do projecto: Macro F1 = 0.623 face a 0.42 do MLP plano.

7) *MLP Hierárquico de 3 Estágios* (*ThreeStageHierarchical.py*): O modelo de 2

estágios já isola a decisão binária da magnitude. O problema remanescente no Estágio 2 é que um único regressor cobre o intervalo $[15, +\infty)$ misturando dois regimes de distribuição muito distintos: o aglomerado denso e estreito 15–30 min e a cauda longa e pesada dos atrasos prolongados. A proposta do docente estende a cascata a **três estágios** implementados na classe `ThreeStageHierarchical`:

- **Estágio 1:** classificador triclasse nativo (softmax 3 saídas), mesmo backbone, `class_weight='balanced'`, `MacroF1Monitor`, divisão de validação estratificada por classe triclasse.
- **Estágio 2a:** regressor treinado **exclusivamente** nos voos Short do treino (`y_train_3c == 1`, alvo $\in [15, 30]$ min). *EarlyStopping(val_loss)*, loss MSE. O alvo estreito torna o MSE mais fácil de minimizar.
- **Estágio 2b:** regressor treinado **exclusivamente** nos voos Long (`y_train_3c == 2`, alvo > 30 min). Especializado na cauda pesada, sem interferência do aglomerado 15–30 min.
- **Inferência:** o Estágio 1 encaminha cada voo: *On-time* → prediz 0 min; *Short* → Estágio 2a; *Long* → Estágio 2b.

A avaliação de cada regressor é feita sobre o **subconjunto verdadeiro** do teste (`y_test_3c == 1` ou `== 2`), isolando a qualidade intrínseca de cada regressor do erro de classificação do Estágio 1.

Tabela XI: 3 Estágios — Métricas por Componente e Ponta-a-Ponta

Componente	RMSE	MAE	R ²	Macro F1	Acc.
Estágio 1 (3cl.)	—	—	—	0.33	45.8%
Estágio 2a (Short)	6.03	5.10	-0.754	—	—
Estágio 2b (Long)	124.75	62.23	-0.324	—	—
E2E cascata	58.72	26.09	—	—	—

O resultado mais relevante é o Estágio 2a: **MAE = 5.10 min, RMSE = 6.03 min** sobre os voos verdadeiramente Short do teste — um ganho de $\approx 9\times$ face ao regressor único do modelo de 2 estágios (MAE = 45.73 min). A especialização é a única causa do ganho: o modelo 2a nunca vê um único voo pontual ou de longo atraso, pelo que o seu gradiente é guiado exclusivamente pela distribuição estreita $[15, 30]$. O Estágio 2b, apesar de mais especializado, atinge RMSE = 124.75 min: a cauda longa tem variância muito elevada — atrasos de 30 min a 500+ min — e nenhuma *feature* pré-partida captura os eventos que a causam.

O custo da cascata é o **erro de encaminhamento composto**: um voo *Long* verdadeiro classificado como *Short* pelo Estágio 1 é enviado ao regressor 2a, cuja saída está limitada a ≈ 30 min, produzindo erro sistemático. O MAE de ponta a ponta (26.09 min) reflete este erro composto e é significativamente pior do que as métricas por subconjunto verdadeiro.

Nota sobre o R^2 negativo do Estágio 2a: O facto de $R^2 = -0.754$ ser negativo mesmo com MAE = 5.10 min é aparentemente paradoxal. A explicação: o R^2 compara o MSE do modelo com o MSE de um preditor da média da *amostra de avaliação*. A média dos atrasos Short verdadeiros é ≈ 22

min, e a variância intra-classe é suficientemente grande para que prever a média seja melhor (em MSE) do que qualquer regressão baseada nas *features* pré-partida. O modelo tem erros médios baixos (MAE = 5.10) mas não consegue superar o preditor trivial em MSE — mais uma evidência do piso de ruído irredutível.

E. Agrupamento de Aeroportos (*Clustering.py*)

1) *Entidade, Agregação e Features*: O clustering foi aplicado a **aeroportos** (entidade operacional natural): um aeroporto é caracterizado pelo perfil agregado de todos os voos que origina — não faz sentido clusterizar voos individuais, pois cada voo é um evento independente sem identidade persistente. A classe `Clustering` recebe o **checkpoint de dados limpos** (`data_loader_cleaned.pkl`, antes da codificação) para que a coluna `ORIGIN` ainda contenha os códigos IATA em texto, necessários para o `groupby`.

A agregação usa os primeiros 300.000 voos (`sample_size=300000`) e filtra aeroportos com menos de 30 voos (`min_flights=30`), retendo **306 aeroportos**. A matriz de *features* resultante tem **13 colunas** (Tabela XII), cobrindo quatro dimensões operacionais: volume, desempenho de pontualidade, perfil de rota e padrão temporal. Todas as *features* são normalizadas com `StandardScaler` antes do clustering — condição essencial para que a distância Euclidiana e a distância de Ward (ambas usadas pelos algoritmos seguintes) não sejam dominadas por variáveis de grande amplitude.

Tabela XII: Features de aeroporto usadas no clustering

Feature	Definição
<code>n_flights</code>	Número total de voos de origem
<code>mean_arr_delay</code>	Média de <code>ARR_DELAY</code>
<code>std_arr_delay</code>	Desvio-padrão de <code>ARR_DELAY</code>
<code>median_arr_delay</code>	Mediana de <code>ARR_DELAY</code>
<code>mean_distance</code>	Distância média das rotas
<code>mean_crs_elapsed</code>	Tempo de voo planeado médio
<code>pct_ontime</code>	Fracção de voos <i>On-time</i>
<code>pct_short</code>	Fracção de voos <i>Short delay</i>
<code>pct_long</code>	Fracção de voos <i>Long delay</i>
<code>pct_weekend</code>	Fracção de voos ao fim de semana
<code>pct_holiday_month</code>	Fracção de voos em mês de férias
<code>n_destinations</code>	Número de destinos distintos
<code>n_airlines</code>	Número de companhias distintas

2) *K-Means — Varredura sobre k* (`kmeans_sweep`): Para seleccionar o número de *clusters*, o K-Means foi executado para $k \in [2, 8]$ com `n_init=10` reinicializações aleatórias por valor de k (para mitigar a sensibilidade ao centroide inicial) e `random_state=42`. Quatro métricas foram calculadas simultaneamente: inércia (*elbow plot*), Silhouette (coesão vs separação, máximo = melhor), Davies-Bouldin (rácio intra/inter-cluster, mínimo = melhor) e Calinski-Harabász (*ratio of between-to- within cluster scatter*, máximo = melhor).

Cluster 0 — Regional/Lazer (35 aeroportos): aeroportos de resort e destinos de montanha como Aspen (ASE) e Jackson Hole (JAC). O desvio-padrão de atraso ($\sigma = 117$ min) é o mais elevado dos três clusters, reflectindo a exposição mete-

Tabela XIII: K-Means — Métricas por k (306 aeroportos)

k	Inércia	Silhouette	D.-Bouldin	C.-Harabász
2	3 233	0.193	1.845	70
3	2 689	0.208[†]	1.501	73[†]
4	2 371	0.185	1.490	68
5	2 152	0.195	1.454	64
6	2 006	0.169	1.667	59
7	1 893	0.189	1.550	55
8	1 786	0.184	1.559	52

[†]Máximo — $k = 3$ selecionado pelo cotovelo e Silhouette.

Tabela XIV: K-Means $k = 3$ — Perfis Operacionais

C	N	Top aeroportos	$\bar{x}$ atraso	σ	Pontual
0	35	ASE, PGD	JAC, 44.9 min	117	65%
1	152	HNL, LGB	BUR, 19.0 min	55	78%
2	119	ATL, ORD	DFW, 20.5 min	60	74%

orológica extrema destes aeroportos e a sua menor resiliência operacional. A taxa de pontualidade de 65% é a mais baixa.

Cluster 1 — Mainstream Médio (152 aeroportos): aeroportos de dimensão intermédia — incluindo Honolulu (HNL), Burbank (BUR) e Long Beach (LGB) — com a melhor taxa de pontualidade (78%) e desvio-padrão mais baixo ($\sigma = 55$ min). Este cluster representa a maioria dos aeroportos regionais dos EUA com operação disciplinada.

Cluster 2 — Grandes Hubs (119 aeroportos): inclui ATL, DFW e ORD, com média de 2.280 voos por aeroporto. O atraso médio (20.5 min) é apenas marginalmente superior ao Cluster 1, demonstrando que o volume massivo de tráfego não implica necessariamente maior impuntualidade — a eficiência operacional à escala é mensurável nos dados.

3) *DBSCAN* — Varredura sobre ε (*dbscan_sweep*): O DBSCAN [7] foi executado com `min_samples=5` e varredura sobre $\varepsilon \in \{0.50, 0.75, 1.00, 1.25, 1.50, 1.75, 2.00, 2.50, 3.00\}$ no espaço escalonado. A métrica de Silhouette é calculada apenas sobre os pontos *não-classificados como ruído* (etiqueta $\neq -1$), com pelo menos 2 *clusters* formados.

Tabela XV: DBSCAN — Varredura sobre ε (`min_samples = 5`)

ε	N. Clusters	N. Ruído	% Ruído	Silhouette
0.50	0	306	100%	—
1.00	2	275	90%	0.223
1.50	2	158	52%	0.275
2.00	1	86	28%	—
3.00	1	23	7.5%	—

No melhor $\varepsilon = 1.5$ (máximo Silhouette), o DBSCAN forma 2 clusters mas classifica **158 de 306 aeroportos (52%) como ruído**. Esta não é uma falha do algoritmo — é um **diagnóstico estrutural** de alta qualidade. O DBSCAN só forma *clusters* em regiões de densidade suficiente ($\geq \text{min_samples}$ pontos

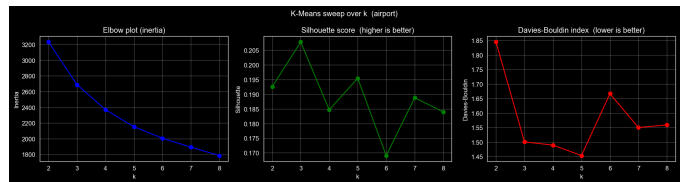


Figura 13: K-Means — varredura sobre k : inércia (cotovelo), Silhouette e Davies-Bouldin. O cotovelo suaviza-se em $k = 3$, que também maximiza o Silhouette.

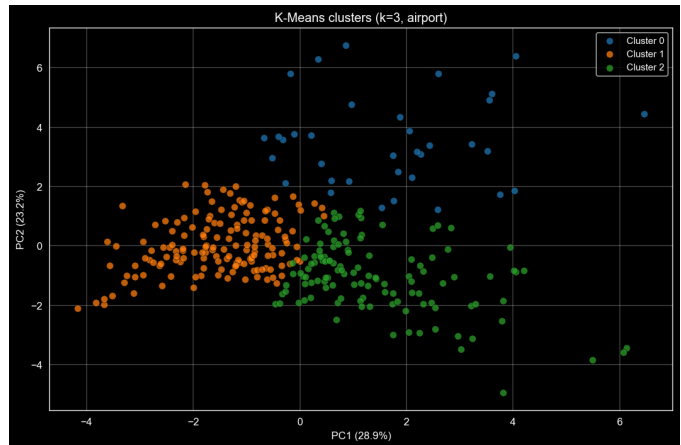


Figura 14: Projeção PCA dos 306 aeroportos colorida pelos 3 clusters K-Means. O Cluster 0 (regional/lazer) separa-se ao longo do PC1 (28.9% da variância), que captura sobretudo o volume de tráfego.

dentro de ε); ao recusar classificar 52% dos pontos, está a identificar que o espaço de *features* dos aeroportos é um **gradiente contínuo**, não um conjunto de aglomerados compactos e separados. Os 148 pontos não-ruído correspondem aos extremos do gradiente — os mega-hubs e os aeroportos de resort — que formam os dois *clusters* com melhor separação. A implicação operacional é que a maioria dos aeroportos dos EUA tem perfis intermédios sem fronteiras nítidas.

4) *Agglomerative Ward* (*agglomerative*, $k = 4$): O clustering hierárquico aglomerativo usa a **ligação de Ward**: em cada passo, fundem-se os dois grupos cuja fusão minimiza o aumento total da variância intra-cluster (equivalente a minimizar a inércia do K-Means). Partindo de 306 *clusters* singulares e fundindo iterativamente, a classe `Clustering.agglomerative` calcula a matriz de ligação completa com `scipy.cluster.hierarchy.linkage(method='ward')` e plota o **dendrograma truncado às últimas 30 fusões** para interpretabilidade.

Com $k = 4$ escolhido após inspeção visual do dendrograma, os resultados são: Silhouette = 0.156, Davies-Bouldin = 1.595. O algoritmo hierárquico [8] **reproduz exactamente** os Clusters 0 e 1 do K-Means e **subdivide o Cluster 2** nos dois sub-grupos que o K-Means fundia numa única categoria: *grande-médio* (≈ 90 aeroportos, 768 voos/aeroporto em média)

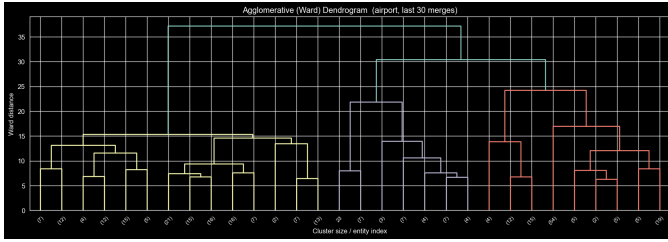


Figura 15: Dendrograma Ward (últimas 30 fusões, 306 aeroportos). A grande distância de ligação entre os dois ramos principais (~37 unidades Ward) confirma a separação nítida do grupo regional/lazer; a segunda grande cisão (~30 unidades) separa médio porte de mega-hubs, correspondendo exactamente à subdivisão do Cluster 2 do K-Means.

e *mega-hubs* (≈31 aeroportos, 6.214 voos/aeroporto). A convergência entre K-Means e Agglomerative Ward — algoritmos com funções objetivo distintas (inércia vs variância aglomerativa) e sem partilha de estado — nas mesmas categorias operacionais é **evidência robusta de que estas categorias são intrínsecas aos dados** e não artefactos da escolha algorítmica.

Tabela XVI: Comparação de Algoritmos de Clustering — Síntese

Algoritmo	N. Clusters	Silhouette	D.-Bouldin	Diagnóstico
K-Means ($k = 3$)	3	0.208	1.501	Partição ótima
DBSCAN ($\epsilon = 1.5$)	2 + 52% ruído	0.275*	—	Gradiente contínuo
Agglomerative ($k = 4$)	4	0.156	1.595	Confirma + subdivisão

*Silhouette calculado apenas nos pontos não-ruído.

VI. FASE 5: COMPARAÇÃO E AVALIAÇÃO DE MODELOS

A. Metodologia de Comparação

Para que a comparação seja justa, todos os modelos são avaliados sobre o **mesmo conjunto de teste** de 50.000 voos, com os mesmos *targets* triclasse derivados pela mesma função de categorização ($<15 \text{ min} \rightarrow 0$; $15\text{--}30 \rightarrow 1$; $>30 \rightarrow 2$). As métricas primárias são o **Macro F1** — imune à dominância da classe maioritária — e o **F1 por classe**, que decompõe o desempenho nos três cenários operacionais distintos. A exatidão global é reportada para contexto mas não é usada como métrica de selecção. Os modelos de regressão são avaliados por **MAE** e **RMSE** (em minutos) e por R^2 (capacidade de superar o preditor trivial da média); dado o piso de ruído irredutível identificado, o MAE ponta-a-ponta é a métrica mais operacionalmente interpretável.

B. Tabela Comparativa Global — Classificação

C. Tabela Comparativa Global — Regressão

D. Análise Multi-Dimensional dos Resultados

a) Dimensão 1 — Efeito da capacidade do modelo.:

A progressão KNN $\rightarrow$ Supervisado $\rightarrow$ Ensemble $\rightarrow$ MLP em Macro F1 ($0.37 \rightarrow 0.37 \rightarrow 0.38 \rightarrow 0.42$) é positiva mas modesta. Nenhum modelo, independentemente da sua capacidade, consegue Macro F1 acima de 0.42 na formulação

Tabela XVII: Comparação Global — Todos os Modelos de Classificação (50k teste)

Modelo	Tipo	Acc.	F1 _{ON}	F1 _{SH}	F1 _{LG}	Macro F1
KNN ($k = 5, 5k$)	3cl	60%	0.72	0.14	0.25	0.37
Árvore Decisão	3cl	55%	0.67	0.17	0.26	0.37
Reg. Logística	3cl	57%	0.70	0.15	0.23	0.36
RF Exp. 1 (original)	3cl	63%	0.76	0.21	0.10	0.35
HGB Exp. 1 (original)	3cl	65%	0.77	0.24	0.13	0.38
RF Exp. 2 (balanceado)	3cl	49%	0.58	0.23	0.28	0.36
HGB Exp. 2 (balanceado)	3cl	51%	0.60	0.25	0.29	0.38
MLP Plano Exp. 1	3cl	70.0%	0.84	0.25	0.17	0.42
MLP 2-Est. (composto)	3cl	57.2%	0.72	0.00	0.34	0.36
MLP 3-Est. (Estágio 1)	3cl	45.8%	0.62	0.17	0.20	0.33
MLP 2-Est. Stage 1	2cl	68.1%	(tarefa binária)			0.623

Tabela XVIII: Comparação Global — Modelos de Regressão (por subconjunto de avaliação)

Modelo / Âmbito de avaliação	de	RMSE	MAE	R^2
MLP plana (50k completo)		303.52	36.40	−28.04
MLP 2-Est. Stage 2 (subconj. atrasados)	2	93.14	45.73	−0.009
MLP 3-Est. Stage 2b (subconj. Long)	2b	124.75	62.23	−0.324
MLP 3-Est. Stage 2a (subconj. Short)	2a	6.03	5.10	−0.754
MLP 3-Est. E2E cascata (50k completo)		58.72	26.09	—

Regressores avaliados no subconjunto verdadeiro correspondente à sua especializ

triclasse — o que indica que o **piso de ruído** das *features* pré-partida está perto de 0.40 Macro F1 para esta tarefa. Mais capacidade computacional não resolve um problema de informação.

b) *Dimensão 2 — Efeito do balanceamento.*: A comparação Ensemble Exp. 1 vs Exp. 2 quantifica o *trade-off* precisamente: o balanceamento triclasse ganha +0.15–0.17 no F1 de *Long* mas perde −14 p.p. na exatidão global. O Macro F1 mantém-se estável (± 0.01), confirmando que é a métrica certa para capturar este *trade-off* sem viés.

c) *Dimensão 3 — arquitetura plana vs hierárquica.*: O MLP plano triclasse (Macro F1 = 0.42) supera o Estágio 1 do modelo de 3 estágios (Macro F1 = 0.33). Paradoxalmente, a formulação *hierárquica* piora a classificação triclasse: o Estágio 1 do modelo de 3 estágios tem uma tarefa mais difícil (triclasse com desequilíbrio 73/10/17) e recebe sinal de paragem pelo Macro F1 triclasse, que é um objetivo mais ruidoso do que o Macro F1 binário. A formulação hierárquica *ganha* na regressão (MAE ponta-a-ponta 26 min vs 36 min plana) e na questão binária (Macro F1 = 0.623).

d) *Dimensão 4 — A classe Short como diagnóstico transversal.*: O F1 de *Short delay* é o mais baixo em todos os modelos, variando entre 0.00 e 0.25. A janela de 15

min (15–30) é intrinsecamente estreita: o MAE irreduzível da regressão (≈ 26 min) é **mais largo do que a janela inteira**. Isto explica mecanicamente por que a classificação de *Short* é impossível ao nível de precisão operacional: qualquer modelo que preveja “vai atrasar” erra sistematicamente na magnitude, distribuindo erros entre *Short* e *Long*.

E. Forças e Fraquezas por Família de Modelos

a) *KNN*: Força: sem parâmetros de treino, interpretável, evidência da eficácia da escalonagem. Fraqueza: $O(N \cdot d)$ por inferência — inviável em produção nesta escala sem KD-Tree ou Ball-Tree. Serve exclusivamente como *baseline* e prova de conceito.

b) *Árvore de Decisão / Regressão Logística*: Força: interpretáveis e auditáveis (a árvore produz regras de decisão explícitas). Fraqueza: a Árvore sem profundidade limitada sobre-ajusta; a Regressão Logística é limitada a fronteiras lineares num problema intrinsecamente não-linear. Ambos ficam abaixo do HGB em Macro F1.

c) *Random Forest / HistGradientBoosting*: Força: robustos e escaláveis; o HGB é o melhor modelo sklearn com Macro F1 estável em ambos os regimes de balanceamento. O RF é paralelizável. Fraqueza: o treino sequencial do HGB limita a paralelização; ambos são caixas negras face à Árvore.

d) *MLP Plano*: Melhor classificador triclasse (Macro F1 = 0.42). O `MacroF1Monitor` é a contribuição técnica central — sem ele o modelo colapsa para o preditor maioritário. Fraqueza: sensível a hiperparâmetros e requer GPU para treino em larga escala; três épocas de convergência variam entre execuções.

e) *MLP Hierárquico 2 Estágios*: Melhor classificador binário do projecto (Macro F1 = 0.623). A questão «vai atrasar?» é a mais accionável e a que tem mais sinal nas *features* pré-partida. O composto triclasse (Macro F1 = 0.36) é inferior ao MLP plano porque o *binning* da saída do regressor é impreciso — a fronteira de 30 min não corresponde à distribuição real dos dados.

f) *MLP Hierárquico 3 Estágios*: Melhor MAE de ponta a ponta (26.09 min) e melhor RMSE no subconjunto Short (6.03 min). Desvantagem: o erro composto de encaminhamento afecta o MAE ponta-a-ponta; mantém três modelos independentes em produção.

F. Limitação Fundamental: Piso de Ruído Irreduzível

O resultado negativo mais informativo do projecto é a negatividade universal de R^2 em todos os cenários de regressão. Mesmo o Stage 2a (intervalo alvo de apenas 15 min, $R^2 = -0.754$) e o Stage 2 do modelo de 2 estágios (treinado *exclusivamente* em voos atrasados, $R^2 = -0.009$) são incapazes de superar o preditor da média. Isto constitui **prova empírica** de que a variância do atraso ao minuto é dominada por variáveis ausentes do dataset:

- **Meteorologia em tempo real**: visibilidade, vento, precipitação na janela de partida/chegada — ausente do dataset mas responsável por >50% dos atrasos de longa duração.

- **Atrasos em cascata por aeronave**: o histórico do avião nas horas anteriores (número de cauda, atraso acumulado) — não incluído nas *features* pré-partida.
- **Congestionamento ATC**: capacidade do espaço aéreo e atrasos na torre de controlo — dinâmico ao minuto e ausente do dataset.

O MAE irreduzível de ≈ 26 min é mais largo do que toda a banda *Short delay* (15 min), o que explica mecanicamente o baixo F1 desta classe em todos os modelos e motiva a priorização da classificação binária como produto operacional principal.

VII. FASE 6: OPERACIONALIZAÇÃO

A. Seleção do Modelo para Produção

A Fase 5 produziu resultados complementares que justificam diferentes modelos para diferentes casos de uso operacional. A Tabela XIX resume as recomendações com os requisitos de cada cenário.

Tabela XIX: Recomendações por caso de uso operacional

Caso de uso	Modelo	Macro F1	MAE
Alerta binário (vai atrasar?)	MLP 2-Est. Estágio 1	0.623	—
Triclasse (severidade)	MLP Plano Exp. 1	0.42	—
Minutos esperados	MLP 3-Est. E2E	—	26 min
Subconjunto Short	MLP 3-Est. Estágio 2a	—	5.1 min

O **produto operacional primário** é o **Estágio 1 binário** (Macro F1 = 0.623, exatidão 68.1%): a questão «vai atrasar?» tem o maior sinal nas *features* pré-partida e é a mais accionável para as decisões de negócio (alocação de portões, alertas proativos a passageiros, reforço de tripulação). O utilizador final recebe um sinal claro de risco sem a ambiguidade da granularidade triclasse.

Para granularidade triclasse (e.g. priorização de portões por severidade), o **MLP Plano Exp. 1** (Macro F1 = 0.42) é preferível ao composto do modelo de 2 estágios (Macro F1 = 0.36) porque não depende da qualidade do regressor de minutos. Quando a estimativa quantitativa de atraso é necessária (e.g. planeamento de conexões), o **modelo de 3 estágios** é o único com MAE abaixo de 30 min (26.09 min ponta-a-ponta). Em qualquer cenário, o utilizador deve ser informado do limite fundamental: a previsão ao minuto tem erro médio de ≈ 26 min independentemente do modelo.

B. Plano de Deployment e arquitetura de Produção

- 1) **Serialização dos artefactos de modelo**. Os três Estágios do modelo de 3 estágios são serializados separadamente em formato `.keras` (TensorFlow Saved-Model). Os transformadores de pré-processamento (*LabelEncoder*, *StandardScaler*, *MinMaxScaler* e o mapa `hourly_delay_map`) são serializados em `.pkl` via `joblib`. Todos os artefactos devem ser versionados em

conjunto no repositório GitHub com *tags* semânticas (`model-v1.0.0`) para rastreabilidade.

- 2) **Serviço de inferência via API REST.** A *pipeline* é exposta como endpoint REST com FastAPI. O *input* de cada pedido é um objecto JSON com as *features* pré-partida: companhia aérea, aeroporto de origem e destino, horário planeado de partida e chegada, distância e data. O serviço aplica toda a *pipeline* de engenharia e pré-processamento (inclusive o `AVG_DELAY_PER_HOUR`) e devolve a probabilidade de atraso, a classe predita e, opcionalmente, a estimativa de minutos. A separação do serviço de inferência do serviço de treino garante que atualizações de modelo não interrompem o serviço em produção.
- 3) **Re-aplicação rigorosa do pré-processamento.** Os *LabelEncoders* e *Scalers* ajustados no treino devem ser carregados e aplicados **exactamente** a cada pedido de inferência — nunca re-ajustados em produção sobre novos dados, o que alteraria o espaço de *features* e invalidaria os pesos do modelo. O `AVG_DELAY_PER_HOUR` é o único componente que deve ser recalculado periodicamente (e.g. mensalmente) sobre dados recentes para capturar *drift* nos padrões horários de atraso.
- 4) **Re-treino periódico.** Re-treino trimestral com dados recentes do *On-Time Performance Reporting System* do DOT, mantendo o mesmo protocolo de *pipeline* (divisão 80/20, mesmos parâmetros). A re-validação pré-deploy deve comparar o Macro F1 do novo modelo com o do modelo em produção; apenas deploy se houver ganho de ≥ 0.01 Macro F1 ou redução de ≥ 1 min MAE.
- 5) **Monitorização em produção.** Monitorizar três tipos de sinal: (i) *drift* nas distribuições das *features* de entrada (novas rotas, novas companhias, mudanças sazonais); (ii) degradação do Macro F1 em dados de produção rotulados a posteriori (quando os voos aterram, o `ARR_DELAY` real fica disponível); (iii) *concept drift* nos padrões operacionais (e.g. novas políticas da FAA, efeitos pós-pandemia).

C. Melhorias Futuras

a) *Features do dia-de — impacto máximo esperado.*

A Fase 5 demonstrou que a variância residual é dominada por eventos do dia ausentes do dataset. As três *features* com maior potencial de redução do piso de ruído são: (i) previsão meteorológica na janela de partida/chegada (visibilidade, precipitação, vento cruzado) — responsável por $>50\%$ dos atrasos de longa duração; (ii) histórico recente da aeronave (número de cauda) — captura atrasos em cascata que o modelo não consegue prever sem saber de onde vem o avião; (iii) capacidade do espaço aéreo prevista (*NAS delay forecast*). Juntas, estas *features* poderiam elevar o R^2 da regressão para território positivo.

b) *Enriquecimento com clustering.*: O perfil de cluster do aeroporto de origem (regional/lazer, médio, hub, mega-hub) identificado na Fase 4 é uma *feature* categórica pronta a incorporar nos modelos preditivos. O Cluster 0 (regional/la-

zer), com $\sigma = 117$ min, sinaliza uma variabilidade operacional muito superior aos restantes e poderia melhorar as previsões para esse sub-segmento.

c) *Modelos especializados por contexto.*: Treinar modelos separados por companhia aérea ou por corredor de tráfego (e.g. costa-a-costa vs regional) permitiria capturar padrões operacionais idiossincráticos. O modelo global não consegue aprender, por exemplo, que a Southwest tem padrões de pontualidade sistematicamente diferentes da Delta nos mesmos corredores.

d) *Escalabilidade do KNN.*: Substituir a pesquisa exaustiva por KD-Tree (`sklearn.neighbors.KDTree`) ou Ball-Tree reduziria a complexidade de inferência de $O(N \cdot d)$ para $O(\log N \cdot d)$, permitindo avaliação no conjunto de teste completo de 50.000 pontos e comparação directa com os restantes modelos.

e) *Otimização do threshold do Estágio 1.*: O threshold de 0.5 usado no Estágio 1 binário foi fixo. Uma curva ROC com validação cruzada poderia encontrar o threshold ótimo para diferentes funções de custo operacional — por exemplo, um threshold mais baixo prioriza identificar todos os atrasos reais (maior *recall*) a custo de mais falsos positivos.

VIII. CONCLUSÃO

Este projeto percorreu o ciclo de vida completo de Data Science — da preparação e validação estatística dos dados (Fases 1–3) à construção, avaliação e operacionalização de modelos (Fases 4–6) — aplicado à previsão de atrasos de voos comerciais domésticos nos EUA.

A **fase de preparação** provou ser o pilar do projeto: a engenharia de 12 *features* contextuais, com destaque para o *target encoding* horário, foi validada não só pela separação visual no UMAP mas, de forma rigorosa, pelos testes de hipóteses ($p < 0,0001$ em todas as variáveis). A falha do PCA contrastada com o sucesso do UMAP confirmou que o dataset é altamente estruturado mas estritamente não-linear, antecipando o desempenho dos modelos baseados em árvores e redes neuronais.

O **melhor classificador** é o Estágio 1 binário do MLP hierárquico de 2 estágios (Macro F1 = 0.623), com um *design* operacionalmente motivado. O **melhor resultado de regressão** é o MAE = 5.10 min do Estágio 2a do modelo de 3 estágios no subconjunto *Short delay* — um ganho de $9\times$ face ao regressor único.

O **resultado negativo** mais relevante é igualmente preciso: a previsão ao minuto de `ARR_DELAY` é fundamentalmente impraticável com as *features* pré-partida ($R^2 < 0$ em todos os cenários). O MAE irreduzível de ≈ 26 min quantifica o piso de ruído imposto pelos eventos do dia ausentes do dataset e motiva a priorização da formulação como classificação, e do Estágio 1 binário como produto operacional.

Por fim, o diagnóstico e correção do *bug* estrutural `class_weight × EarlyStopping` — com o `MacroF1Monitor` como solução mínima e suficiente — constitui uma contribuição técnica diretamente aplicável a qualquer problema de classificação multiclasse desequilibrada

em Keras. A fase de *clustering* identificou três categorias operacionais intrínsecas aos dados (regional/lazer, mainstream-médio, grandes hubs), convergentemente detetadas por K-Means e Agglomerative Ward.

REFERÊNCIAS

- [1] U.S. Department of Transportation, “Flight Delay and Cancellation Dataset (2019–2023),” Kaggle. [Online]. Disponível: <https://www.kaggle.com/datasets/patrickzel/flight-delay-and-cancellation-dataset-2019-2023/data>
- [2] M. Abadi *et al.*, “TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems,” 2015. [Online]. Disponível: <https://www.tensorflow.org>
- [3] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [4] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001.
- [5] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [6] J. MacQueen, “Some Methods for Classification and Analysis of Multivariate Observations,” *Proc. 5th Berkeley Symp. on Mathematical Statistics and Probability*, vol. 1, pp. 281–297, 1967.
- [7] M. Ester, H.-P. Kriegel, J. Sander e X. Xu, “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise,” *Proc. KDD*, pp. 226–231, 1996.
- [8] J. H. Ward, “Hierarchical Grouping to Optimize an Objective Function,” *Journal of the American Statistical Association*, vol. 58, no. 301, pp. 236–244, 1963.
- [9] T. Cover e P. Hart, “Nearest Neighbor Pattern Classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.